

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**
2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**
3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Code of Conduct:

*I B.VANI(192472348) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: B.VANI

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. RL Framework: Agent–Environment Interaction

Reinforcement Learning (RL) is a learning framework in which an **agent** interacts with an **environment** to learn the best decisions. At each time step, the agent observes a **state (S)**, selects an **action (A)** according to its policy, and receives a **reward (R)** from the environment. The environment then moves to a new state.

The main components are:

- **State:** Represents the current condition of the environment.
- **Action:** A decision that can be taken by the agent.
- **Reward:** Numerical feedback indicating how good or bad an action was.
- **Policy:** Strategy used by the agent to select actions.

The agent attempts to maximize the **cumulative reward** rather than only the immediate reward. The return is generally represented as:

$$[G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots]$$

where (γ) is the discount factor. Therefore, an action that provides a smaller immediate reward but larger future benefits may be preferred.

2. Decision-Making as a Markov Decision Process

A **Markov Decision Process (MDP)** provides a mathematical framework for modeling sequential decision-making problems. An MDP is represented as:

$$[MDP = (S, A, P, R, \gamma)]$$

where:

- **S:** Set of possible states.
- **A:** Set of possible actions.
- **P(s'|s,a):** Transition probability of reaching state (s') from state (s) after action (a).
- **R(s,a):** Reward obtained after taking an action.
- **γ :** Discount factor that determines the importance of future rewards.

For example, in a robot navigation problem, the current location is the state, movement directions are actions, movement to another location represents the transition, reaching the destination provides a positive reward, and collisions provide negative rewards.

3. Policy and Value Functions

A **policy ($\pi(a|s)$)** defines the probability of selecting action (a) when the agent is in state (s).

The **state-value function** ($V^\pi(s)$) represents the expected cumulative reward obtained by following policy (π) from state (s):

$[V^\pi(s) = E_\pi[G_t | S_t = s]]$ The **action-value function** ($Q^\pi(s, a)$) represents the expected cumulative reward when action (a) is taken in state (s), followed by policy (π):

$[Q^\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]]$

The Bellman equation for the state-value function is:

$[V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]]$

For the action-value function:

$[Q^\pi(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]]$

Thus, value functions help the agent estimate the long-term benefit of different states and actions.

4. Exploration–Exploitation Trade-Off

The exploration–exploitation trade-off is the problem of deciding whether an agent should **explore new actions** or **exploit the best-known action**.

ϵ -Greedy Strategy: With probability (ϵ), the agent selects a random action for exploration. With probability ($1 - \epsilon$), it selects the action with the highest estimated value.

Softmax Strategy: Actions are selected according to probabilities based on their estimated values. Higher-value actions receive higher probabilities, while lower-value actions still have a chance of being selected.

ϵ -Greedy

Simple to implement

Random exploration

Easy to control using ϵ

May select a very poor action randomly

Softmax

Provides graded action probabilities

Exploration depends on action values

Controlled using temperature

Usually provides smoother exploration

Both strategies help the agent discover better policies while still using previously learned knowledge.

5. Reward Shaping in Reinforcement Learning

Reward shaping involves providing additional intermediate rewards to guide the agent toward the desired behavior.

For example, in a robot navigation problem, the agent can receive:

- Positive reward for moving closer to the destination.
- Negative reward for moving away from the destination.
- Large positive reward for reaching the destination.
- Large negative reward for collisions.

Without reward shaping, the agent may receive a reward only after completing a long sequence of actions, making learning slow. Appropriate intermediate rewards provide more frequent feedback and accelerate learning.

Reward shaping can also prevent suboptimal policies by encouraging useful intermediate behaviors. However, poorly designed rewards may cause the agent to optimize the wrong objective.

6. Challenges of RL in Real-World Robotics

Applying RL to real-world robotics presents several challenges.

Safety: Incorrect actions can cause collisions, equipment damage, or injury. Safety constraints and emergency mechanisms are therefore required.

Sample Efficiency: RL may require a large number of interactions to learn an effective policy. Performing all these interactions on a physical robot can be expensive and time-consuming.

Environment Variability: Real environments change due to lighting, obstacles, sensor noise, temperature, and unexpected objects. A policy trained in one environment may not perform equally well in another.

Therefore, simulation, transfer learning, safety constraints, and real-world validation are important for reliable robotic RL systems.

7. Effect of Discount Factor (γ)

The discount factor (γ) determines the importance assigned to future rewards.

$$[G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots]$$

- If (γ) is close to **0**, the agent focuses mainly on immediate rewards.
- If (γ) is close to **1**, the agent gives greater importance to future rewards.

For example, if ($\gamma=0.2$), immediate rewards are strongly preferred. If ($\gamma=0.9$), the agent is more likely to choose actions that provide long-term benefits.

In episodic tasks, the discount factor helps determine whether the agent prioritizes immediate results or the overall outcome of the episode.

8. Model-Free RL vs Model-Based RL

Model-Free RL learns a policy or value function directly from interactions without explicitly learning the environment's transition model.

Examples include **Q-learning** and **SARSA**.

Model-Based RL learns or uses a model of the environment, including transition probabilities and rewards, and uses this model for planning.

Model-Free RL

Does not require an environment model

Learns directly from experience

Generally simpler

Can require many interactions

Suitable for complex unknown environments

Model-Based RL

Requires or learns an environment model

Uses planning and prediction

More computationally complex

Can be more sample-efficient

Useful when the environment can be modeled

In dynamic environments, model-free RL can adapt directly from experience, while model-based RL can use its learned model to plan future actions. Model-based methods may suffer when the learned model is inaccurate.

9. Q-Learning Update and Convergence

Q-learning is an **off-policy, model-free RL algorithm** that learns the optimal action-value function.

The update equation is:

$$[Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]]$$

Consider:

$$[Q(s,a)=2, \quad R=5, \quad \gamma=0.9, \quad \alpha=0.5]$$

Suppose:

$$[\max_{a'} Q(s', a') = 6]$$

Then:

$$[Q_{\text{new}}(s, a) = 2 + 0.5[5 + (0.9 \times 6) - 2]] \quad [= 2 + 0.5[5 + 5.4 - 2]]$$

$$[= 2 + 0.5(8.4)]$$

$$[= 2 + 4.2 = 6.2]$$

Therefore:

$$[\boxed{Q_{\text{new}}(s, a) = 6.2}]$$

During training, Q-values are repeatedly updated using new experiences. Initially, the values may be inaccurate, but with sufficient exploration and repeated updates, the Q-values can converge toward the optimal action values. Consequently, the agent learns an increasingly effective policy.

10. On-Policy vs Off-Policy Learning

On-policy learning learns the value of the same policy that the agent is currently following. An example is **SARSA**.

Off-policy learning learns an optimal or target policy while potentially following a different behavior policy. **Q-learning** is a common example.

On-Policy

Learns from the policy being followed

Example: SARSA

Considers the actual behavior of the agent

Often useful when behavior safety matters

Off-Policy

Can learn a different target policy

Example: Q-learning

Can learn optimal behavior independently

Useful for learning from different experiences

Advantages of On-Policy: It can learn policies that account for the agent's actual behavior and exploration strategy.

Advantages of Off-Policy: It can reuse experiences generated by different policies and can learn an optimal policy while following an exploratory behavior policy.

Thus, the choice between on-policy and off-policy learning depends on the application, safety requirements, exploration strategy, and availability of experience data.